Аннотация

Содержание

1 Введение **5**

1.1 Системное программирование 5

1.2 Теория типов 6

1.2.1 Зависимые типы 7

1.3 Другие системные языки программирования 7

1.3.1 C 7

1.3.2 C++ 8

1.3.3 Rust 9

1.3.4 ATS 10

1.4 Постановка задачи 10

2 Лямбда-исчисление с зависимыми типами **11**

2.1 Система типов 11

2.2 Метатеоремы 16

2.3 Задача полной и частичной типизации 16

3 Операционная семантика **17**

3.1 Семантика лямбда-исчисления 17

3.2 Стековая машина с кодовой секцией 17

3.3 Машина с кучей 17

4 Линейные типы и ресурсная логика **18**

4.1 Безопасное управление памятью 18

4.2 Линейные зависимые типы 18

5 Экзистенциальные типы **20**

5.1 Эффективное представление замыканий 20

5.2 Вселенные 20

5.3 Рефлективные подвселенные 20

6 Прямой доступ к памяти **21**

6.1 Прямой доступ к стеку 21

6.2 Прямой доступ к куче 21

7 Защищённая рекурсия **22**

8 Исчисление указателей **23**

8.1 Логический фреймворк 23

8.2 Метатеоремы 23

8.3 Алгоритм частичной типизации 23

9 Дальнейшие расширения **24**

9.1 Строковый полиморфизм 24

9.2 Полиморфизм уровней 24

9.3 Аллокации арен 24

9.4 Многопоточное исполнение и модель памяти 24

9.5 Двухуровневая система типов 24

10 Заключение **25**

1 Введение

1.1 Системное программирование

Системное программирование — процесс создания программ, решающих задачи общевычислительного характера, таких как выделение и разделение ресурсов, доступ к устройствам, обеспечение среды для разработки, запуска и выполнения других программ. [12] Кроме относящейся сюда разработки операционных систем, интегрированных сред разработки и систем управления базами данных, часто к системному программированию также относят разработку высокопроизводительных серверов и программных утилит, напрямую работающих с ресурсами компьютера, а также программирование встраиваемых систем. [1]

По прошествии изначального этапа становления программирования и компьютерных наук как самостоятельной инженерной и научной дисциплины, интерес к системному программированию угас в силу экспоненциального роста производственных мощностей благодаря многократному увеличению количества транзисторов, которые физически можно расположить в микросхеме процессора (так называемый закон Мура [3]). Однако в последнее время производители столкнулись с фундаментальными преградами на пути уменьшения размеров резисторов (на уровне 5нм ощутимые помехи начинают создавать квантовые эффекты), так что постоянно растущие потребности в вычислительной мощности больше нельзя компенсировать покупкой более мощного оборудования, и к системному программированию вернулся оживлённый интерес. Более того, как уже упоминалось выше, технологии системного программирования неизбежно применяются в сфере программирования встраиваемых систем, чрезвычайно популярных в последнее время благодаря введению в моду концепций Умного дома [4] и Интернета вещей [7].

В силу специфичной сферы применения, к программам, разрабатываемым в рамках системного программирования, предъявляются повышенные требования к быстродействию, экономности по памяти и одновременно безопасности времени исполнения — отсутствие критических ошибок, приводящих к отказу работы других программ либо к потере пользовательских данных. Однако зачастую эти требования оказываются противоречащими друг другу, так что ландшафт решений для облегчения работы системного программиста состоит из всевозможных компромиссов и поэтому крайне широк:

- Библиотеки для ручного управления памятью в высокоуровневых языках

программирования [8] [13];

- Библиотеки, абстрагирующие управление памятью в низкоуровневых языках программирования [5];
- Библиотеки, предоставляющие высокоуровневый интерфейс взаимодействия с внешней средой в низкоуровневых языках программирования [6];
- Утилиты для слежения за некорректным обращением с памятью [9];
- Дебаггеры для слежения за исполнением программы [2];
- Статические анализаторы системных языков программирования для поиска ошибок в обращении с памятью (и других возможных ошибок) с помощью абстрактной интерпретации [11];
- Собственно, системные языки программирования.

Для нас особый интерес представляют именно языки программирования, поскольку среди вышеперечисленных средств только они могут включать в себя надёжный способ проверки корректности программы в виде специальной **системы типов**.

1.2 Теория типов

Система типов — исполнимый синтаксический метод доказательства отсутствия определённого поведения программы с помощью классификации выражений по типу значений, которые они вычисляют. [10] Кроме этого, заданная правильным образом система типов помогает решать следующие задачи системного программирования:

- Представление данных в памяти процесса;
- Эффективное исполнение программ;
- Контроль побочных эффектов.

Заметим, что под “побочными эффектами” в зависимости от контекста понимают различные вещи. Как правило, они включают в себя взаимодействие с внешним миром, исключения, логирование и изменение внешнего по отношению к процедуре фрагмента памяти; в некоторых контекстах (прежде всего в программировании встраиваемых устройств и операционных систем) побочные эффекты включают в себя и динамические аллокации.

1.2.1 Зависимые типы

Отдельной сферой активных исследований в сфере теории типов является исследование и поиск систем с зависимыми типами, т.е. такими, в которых кроме термов, зависящих от других термов, можно вводить определения типов, точно так же зависящих от термов. Помимо того, что зависимые типы позволяют более точно описывать множества значений, оперируемых программой, корректно построенные системы типов с зависимыми типами естественным образом соответствуют логикам высшего порядка (этот феномен также известен как соответствие Карри-Говарда). С этой точки зрения, программы особых типов можно рассматривать как автоматически проверяемые доказательства соответствующих утверждений; более того, такие системы зачастую позволяют строить доказательства свойств программ, написанных на этом же языке, что делает их очень привлекательными средствами для создания доказуемо корректного программного обеспечения.

В частности, в результате данной работы будет предъявлено лямбда-исчисление с зависимыми типами, поддерживающее контроль динамических аллокаций и эффективную компиляцию в программы, исполнимые на устройствах архитектуры фон Неймана; более того, предъявленное исчисление будет сохранять все удобства функционального программирования для написания простого, читаемого и легко компонируемого кода.

1.3 Другие системные языки программирования

Начиная с 1960х годов, появилось множество системных языков программирования, нашедших свои применения на различных платформах и в различных практических приложениях. Среди них особенно выделяются мультиплатформенные языки, обещающие максимальную производительность и удобство на большом количестве различных архитектур и операционных систем. Все они достаточно сильно отличаются друг от друга в силу различных систем типов, диктующих определённый стиль программирования и предоставляющих различные гарантии безопасности и производительности. Далее приведён краткий обзор наиболее значимых представителей.

1.3.1 C

Повсеместно встречающийся и повсеместно изучаемый системный язык программирования с достаточно простой системой типов, включающей структуры (по сути своей типы-произведения), типы-объединения, числовые типы и типы указателей. В силу

правильным образом подобранных базовых типов и операций, программы на C не требуют никаких накладных расходов по времени и памяти при исполнении программы относительно её исходного кода. Кроме того, простота языка обеспечила ему роль промежуточного языка для общения между участками кода, написанными на различных языках: практически любой другой современный язык программирования поддерживает так называемый Foreign Function Interface с C, то есть возможность вызывать процедуры на C из исходного кода этого языка и наоборот.

С другой стороны, сочетание слабой дисциплины типизации с очень высокой выразительной силой операций (арифметика указателей, прямой доступ к памяти) приводит к огромной свободе действий программиста без возможности автоматически проверить поведение получающихся программ на корректность, что приводит к огромному количеству ошибок; некоторые критические ошибки в коде на C сорокалетней давности находят по сей день.

1.3.2 C++

C++ — мультипарадигмальный язык программирования, изначально созданный Бьярном Страуструпом как расширение C средствами объектно-ориентированного программирования с сохранением фокуса на производительности кода. Выбранный в C++ подход стал известен под названием zero-cost abstraction и, если быть точным, значит, что разработчики языка C++ гарантируют, что если в языке введён какой-либо механизм без возможности его замены пользователем языка, то любой пользователь этого языка не сможет реализовать аналогичный механизм более эффективным способом; более того, сам факт существования этого механизма не должен влиять на производительность кода, его не использующего. Например, код, не использующий механизмы выбрасывания и обработки исключений, выполняется так же быстро и экономно по памяти, как если бы в языке не было этого механизма.

Со временем, в C++ было добавлено всё больше zero-cost abstractions и всё больше возможностей, позволяющих писать более краткий и более выразительный код. Таким образом появился язык с производительностью, максимально близкой к C, но на первый взгляд гораздо более удобный и выразительный для написания кода. Однако ключевые проблемы языка C относительно безопасности написанного кода и каких-либо гарантий поведения не были решены (а во многих случаях и усугублены), так что абсолютное большинство современных высокопроизводительных программ содержит

огромное количество кода на C++, уязвимое к всевозможным ошибкам. Как правило, это не является проблемой, когда упавшую программу можно просто перезапустить (а компьютер с упавшей операционной системой — перезагрузить), но в некоторых областях применения — таких как программное обеспечение суден, медицинское оборудование, защита данных от злоумышленников — ошибка просто недопустима, так что код на C/C++ в данных сферах подвергают всевозможным проверкам, могущим показаться безумными любому внешнему наблюдателю.

1.3.3 Rust

Rust — выпущенный Mozilla в 2015 году новый системный язык программирования, созданный для преодоления проблем с безопасностью, создаваемых использованием C/C++. Внешне похожий на языки из семейства ALGOL (куда относятся и C, и C++), тем не менее Rust начинался как язык-наследник ML, одного из старейших статически типизированных функциональных языков программирования; от ML данный язык перенял типы-суммы и параметрический полиморфизм, которые он с ростом популярности языка привнёс в мейнстрим (например, похожие механизмы в итоге были добавлены в новейшие версии C++).

Однако основной инновацией Rust стало использование аффинной системы типов (системы, в которой запрещено произвольное копирование значений) в сочетании с новым механизмом “заимствования”, задающим правила работы со ссылками на аффинные значения, воспроизводящими доказуемо безопасное подмножество операций с указателями из C. Это позволило писать программы с производительностью, максимально приближенной к производительности программ на C, похожие на программы на C, но вдобавок ко всему доказуемо безопасные по памяти!

Единственным, но достаточно серьёзным ограничением стало очень сильное ограничение выразительной силы указателей на значения и их арифметики: в частности, в Rust очень плохо моделируются типы данных, указывающие сами на себя и, в целом, любые сложные графы зависимостей на указателях. В данных случаях пользователи Rust вынуждены прибегнуть к одному из следующего:

- использовать созданные для конкретных случаев механизмы языка (Pin);
- пересоздавать арифметику указателей вручную с помощью числовой арифметики (generational indices);

- использовать так называемые “умные” указатели, требующие дополнительных расходов по времени и памяти (хотя код на C/C++ их не потребует);
- изменить архитектуру программы на более простую;
- использовать дополнительные конструкции языка для работы с указателями, корректность работы которых система типов гарантировать не может.

1.3.4 ATS

Созданный с той же целью, что и Rust — сделать системное программирование безопасным — ATS предлагает гораздо более сложную и продвинутую систему типов, включающую в себя линейные типы (нельзя ни копировать, ни удалять), возможности для функционального программирования и, наконец, средства для доказательства теорем о корректности исполняемых программ. Однако ATS содержит такое большое количество взаимозаменяемых выразительных средств и похожих друг на друга концепций, наоборот имеющих только свои узкие сферы применения, и так сложен в изучении, что его сложно считать успешным опытом реализации системного языка программирования с продвинутой системой типов.

1.4 Постановка задачи

Познакомившись со сферой системного программирования и изучив существующие языки системного программирования, мы пришли к выводу, что достойной и актуальной задачей является создание **системного языка программирования с зависимыми типами** со следующими качествами:

- предоставляющий сильные гарантии безопасности во время исполнения;
- достаточно гибкий, чтобы выразить больше идиом в работе с указателями;
- позволяющий эффективно скомпилированному коду приблизиться по производительности к существующим системным языкам программирования.

2 Лямбда-исчисление с зависимыми типами

Для знакомства с основными понятиями и техниками при проведении исследований в сфере теории типов, а также с выбранной в данном тексте нотацией, мы кратко (но полно) рассмотрим исчисление, являющееся отправной точкой в любой работе на тему зависимых типов: интуиционистскую теория типов, также “конструктивная теория типов”, также “теория типов с зависимыми типами Мартина-Лёфа”. Рассмотрим её декларативные правила типизации, грамматические конструкции, её свойства в качестве логической системы, а также основные сложности в её практической реализации.

2.1 Система типов

Далее приведена формализация системы как приведена в ncatlab. Она включает в себя следующие логические суждения:

$\Gamma \vdash \tau\,\mathrm{type}$	Формулировка типов
$\Gamma \vdash t : \tau$	Введение и устранение термов
$\Gamma \vdash \sigma \equiv \tau$	Равенство типов
$\Gamma \vdash t \equiv u : t$	Равенство термов
$\vdash \Gamma\ \mathrm{ctx}$	Корректность построения контекста

Корректно построенный контекст определяется как последовательность (уникальных) имён, проаннотированных корректно сформулированными типами:

$$\frac{}{\vdash \cdot\ \mathrm{ctx}}\textsc{Ctx-Empty} \qquad \frac{\vdash \Gamma\ \mathrm{ctx} \qquad \Gamma \vdash \tau\,\mathrm{type}}{\vdash \Gamma, x : \tau\ \mathrm{ctx}}\textsc{Ctx-Cons}$$

Имена являются атомарными термами, а контекст поддерживает ослабление и подстановку:

$$\frac{\Gamma \vdash \tau\,\mathrm{type}}{\Gamma, x : \tau \vdash x : \tau}\textsc{Var} \qquad \frac{\Gamma, \Delta \vdash \Phi \qquad \Gamma \vdash \tau\,\mathrm{type}}{\Gamma, x : \tau, \Delta \vdash \Phi}\mathrm{W} \qquad \frac{\Gamma, x : \tau, \Delta \vdash \Phi \qquad \Gamma \vdash t : \tau}{\Gamma, \Delta[t/x] \vdash \Phi[t/x]}\mathrm{S}$$

Равенство-по-определению является конгруэнтностью, и с его помощью можно конвертировать переменные:

$$\frac{\Gamma \vdash \tau\,\mathrm{type}}{\Gamma \vdash \tau \equiv \tau}\equiv\text{-TYPE-REFL} \qquad \frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t : \tau}{\Gamma \vdash t \equiv t : \tau}\equiv\text{-TERM-REFL}$$

$$\frac{\Gamma \vdash \sigma \equiv \tau}{\Gamma \vdash \tau \equiv \sigma}\equiv\text{-TYPE-SYM} \qquad \frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \tau}{\Gamma \vdash u \equiv t : \tau}\equiv\text{-TERM-SYM}$$

$$\frac{\Gamma \vdash \rho \equiv \sigma \qquad \Gamma \vdash \sigma \equiv \tau}{\Gamma \vdash \rho \equiv \tau}\equiv\text{-TYPE-TRANS}$$

$$\frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \tau \qquad \Gamma \vdash u \equiv v : \tau}{\Gamma \vdash t \equiv v : \tau}\equiv\text{-TERM-TRANS}$$

$$\frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \sigma \qquad \Gamma, x : \sigma, \Delta \vdash \tau\,\mathrm{type}}{\Gamma, \Delta[u/x] \vdash \tau[t/x] \equiv \tau[u/x]}\equiv\text{-TYPE-SUB}$$

$$\frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \sigma \qquad \Gamma, x : \sigma, \Delta \vdash v : \tau}{\Gamma, \Delta[u/x] \vdash v[t/x] \equiv v[u/x] : \tau[u/x]}\equiv\text{-TERM-SUB}$$

$$\frac{\Gamma \vdash \sigma \equiv \tau \qquad \Gamma, x : \sigma, \Delta \vdash \Phi}{\Gamma, x : \tau, \Delta \vdash \Phi}\text{VAR-CONV}$$

Исчисление включает в себя типы зависимых функций, зависимых пар, тип-сумму, пустой тип (также известный как 0-тип), тип с единственным значением (1-тип), тип натуральных чисел и, наконец, тип равенства. Каждый из типов добавляет свои правила формулировки типов, введения и устранения термов, а также специальные правила равенства термов, разделяемые на “правила вычисления” и “правила единственности”. Так, функции, возвращаемый тип которых зависит от значения аргумента, задаются следующими правилами вывода:

$$\frac{\Gamma \vdash \sigma\,\mathrm{type} \qquad \Gamma, x : \sigma \vdash \tau\,\mathrm{type}}{\Gamma \vdash \Pi_{x:\sigma}\tau\,\mathrm{type}}\Pi\text{-F} \qquad \frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash \lambda(x : \sigma).t : \Pi_{x:\sigma}\tau}\Pi\text{-I}$$

$$\frac{\Gamma \vdash f : \Pi_{x:\sigma}\tau \qquad \Gamma \vdash t : \sigma}{\Gamma \vdash ft : \tau[t/x]}\Pi\text{-E} \qquad \frac{\Gamma, x : \sigma \vdash t : \tau \qquad \Gamma \vdash u : \sigma}{\Gamma \vdash (\lambda(x : \sigma).t)u \equiv t[u/x] : \tau[u/x]}\Pi\text{-C}$$

$$\frac{\Gamma \vdash f : \Pi_{x:\sigma}\tau}{\Gamma \vdash f \equiv \lambda(x : \sigma).fx : \Pi_{x:\sigma}\tau}\Pi\text{-U}$$

Аналогично вводится тип пар, в которых тип второй компоненты зависит от значения первой компоненты:

$$\frac{\Gamma \vdash \sigma\,\mathrm{type} \qquad \Gamma, x : \sigma \vdash \tau\,\mathrm{type}}{\Gamma \vdash \Sigma_{x:\sigma}\tau\,\mathrm{type}}\Sigma\text{-F} \qquad \frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau[t/x]}{\Gamma \vdash (t, u) : \Sigma_{x:\sigma}\tau}\Sigma\text{-I}$$

$$\frac{\Gamma \vdash t : \Sigma_{x:\sigma}\tau}{\Gamma \vdash \pi_1 t : \sigma}\Sigma\text{-E}_1 \qquad \frac{\Gamma \vdash t : \Sigma_{x:\sigma}\tau}{\Gamma \vdash \pi_2 t : \tau[\pi_1 t/x]}\Sigma\text{-E}_2 \qquad \frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau[t/x]}{\Gamma \vdash \pi_1(t, u) \equiv t : \sigma}\Sigma\text{-C}_1$$

$$\frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau[t/x]}{\Gamma \vdash \pi_2(t, u) \equiv u : \tau[\pi_1(t, u)/x]}\Sigma\text{-C}_2 \qquad \frac{\Gamma \vdash t : \Sigma_{x:\sigma}\tau}{\Gamma \vdash t \equiv (\pi_1 t, \pi_2 t) : \Sigma_{x:\sigma}\tau}\Sigma\text{-U}$$

Тип-сумма $\sigma + \tau$ — по сути своей дизъюнктное объединение типов σ и τ. 0-тип по определению не содержит элементов; 1-тип — содержит единственный элемент. Натуральные числа поддерживают принцип индукции.

$$\frac{(\forall i)\ \Gamma \vdash \sigma_i \,\mathrm{type}}{\Gamma \vdash \sigma_1 + \sigma_2 \,\mathrm{type}}\,{+}\text{-F} \qquad \frac{\Gamma \vdash t : \sigma_i}{\Gamma \vdash \mathrm{inj}_i\, t : \sigma_1 + \sigma_2}\,{+}\text{-I}_i$$

$$\frac{\Gamma \vdash t : \sigma_1 + \sigma_2 \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash \tau \,\mathrm{type} \qquad (\forall i)\ \Gamma, x_i : \sigma_i \vdash t_i : \tau[\mathrm{inj}_i\, x_i/x]}{\Gamma \vdash \mathrm{elim}^{+}_{x.\tau}\, t(\mathrm{inj}_1\, x_1 \mapsto t_1; \mathrm{inj}_2\, x_2 \mapsto t_2) : \tau[t/x]}\,{+}\text{-E}$$

$$\frac{\Gamma \vdash t : \sigma_i \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash \tau \,\mathrm{type} \qquad (\forall j)\ \Gamma, x_j : \sigma_j \vdash t_j : \tau[\mathrm{inj}_j\, x_j/x]}{\Gamma \vdash \mathrm{elim}^{+}_{x.\tau}(\mathrm{inj}_i)(\mathrm{inj}_1\, x_1 \mapsto t_1; \mathrm{inj}_2\, x_2 \mapsto t_2) \equiv t_i[t/x] : \tau[\mathrm{inj}_i\, t/x]}\,{+}\text{-C}_i$$

$$\frac{\begin{array}{c}\Gamma \vdash t : \sigma_1 + \sigma_2 \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash \tau \,\mathrm{type} \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash u : \tau \\ (\forall i)\ \Gamma, x_i : \sigma_i \vdash t_i : \tau[\mathrm{inj}_i\, x_i/x] \qquad (\forall i)\ \Gamma, x_i : \sigma_i \vdash u[\mathrm{inj}_i\, x_i/x] \equiv t_i : \tau[\mathrm{inj}_i\, x_i/x]\end{array}}{\Gamma \vdash u[t/x] \equiv \mathrm{elim}^{+}_{x.\tau}\, t(\mathrm{inj}_1\, x_1 \mapsto t_1; \mathrm{inj}_2\, x_2 \mapsto t_2) : \tau[t/x]}\,{+}\text{-U}$$

$$\frac{\vdash \Gamma \,\mathrm{ctx}}{\Gamma \vdash \mathbb{0} \,\mathrm{type}}\,\mathbb{0}\text{-F} \qquad \frac{\Gamma \vdash t : \mathbb{0} \qquad \Gamma, x : \mathbb{0} \vdash \sigma \,\mathrm{type}}{\Gamma \vdash \mathrm{elim}^{\mathbb{0}}_{x.\sigma}\, t : \sigma[t/x]}\,\mathbb{0}\text{-E}$$

$$\frac{\Gamma \vdash t : \mathbb{0} \qquad \Gamma, x : \mathbb{0} \vdash \sigma \,\mathrm{type} \qquad \Gamma, x : \mathbb{0} \vdash u : \sigma}{\Gamma \vdash u[t/x] \equiv \mathrm{elim}^{\mathbb{0}}_{x.\sigma}\, t : \sigma[t/x]}\,\mathbb{0}\text{-U} \qquad \frac{\vdash \Gamma \,\mathrm{ctx}}{\Gamma \vdash \mathbb{1} \,\mathrm{type}}\,\mathbb{1}\text{-F} \qquad \frac{\vdash \Gamma \,\mathrm{ctx}}{\Gamma \vdash * : \mathbb{1}}\,1\text{-I}$$

$$\frac{\Gamma \vdash t : \mathbb{1} \qquad \Gamma, x : \mathbb{1} \vdash \sigma \,\mathrm{type} \qquad \Gamma \vdash u : \sigma[*/x]}{\Gamma \vdash \mathrm{elim}^{\mathbb{1}}_{x.\sigma}\, t(* \mapsto u) : \sigma[t/x]}\,\mathbb{1}\text{-E}$$

$$\frac{\Gamma, x : \mathbb{1} \vdash \sigma \,\mathrm{type} \qquad \Gamma \vdash t : \sigma[*/x]}{\Gamma \vdash \mathrm{elim}^{\mathbb{1}}_{x.\sigma}\, *(* \mapsto u) \equiv u : \sigma[*/x]}\,\mathbb{1}\text{-C}$$

$$\frac{\begin{array}{c}\Gamma \vdash t : \mathbb{1} \\ \Gamma, x : \mathbb{1} \vdash \sigma \,\mathrm{type} \qquad \Gamma, x : \mathbb{1} \vdash u : \sigma \qquad \Gamma \vdash v : \sigma[*/x] \qquad \Gamma \vdash u[*/x] \equiv v : \sigma[*/x]\end{array}}{\Gamma \vdash u[t/x] \equiv \mathrm{elim}^{\mathbb{1}}_{x.\sigma}\, t(* \mapsto v) : \sigma[t/x]}\,\mathbb{1}\text{-U}$$

$$\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \mathbb{N}\,\text{type}}\mathbb{N}\text{-F} \qquad \frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash 0 : \mathbb{N}}\mathbb{N}\text{-I}_0 \qquad \frac{\Gamma \vdash t : \mathbb{N}}{\Gamma \vdash St : \mathbb{N}}\mathbb{N}\text{-I}_S$$

$$\frac{\Gamma \vdash t : \mathbb{N} \qquad \Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma \vdash t_0 : \sigma[0/x] \qquad \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x]}{\Gamma \vdash \text{elim}^{\mathbb{N}}_{x.\sigma}\, t(0 \mapsto t_0; Sx, h \mapsto t_S) : \sigma[t/x]}\mathbb{N}\text{-E}$$

$$\frac{\Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma \vdash t_0 : \sigma[0/x] \qquad \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x]}{\Gamma \vdash \text{elim}^{\mathbb{N}}_{x.\sigma}\, 0(0 \mapsto t_0; Sx, h \mapsto t_S) \equiv t_0 : \sigma[0/x]}\mathbb{N}\text{-C}_0$$

$$\frac{\Gamma \vdash t : \mathbb{N} \qquad \Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma \vdash t_0 : \sigma[0/x] \qquad \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x]}{\Gamma \vdash \text{elim}^{\mathbb{N}}_{x.\sigma}(St)(0 \mapsto t_0; Sx, h \mapsto t_S) \equiv t_S[t/x][\text{elim}^{\mathbb{N}}_{x.\sigma}\, t(0 \mapsto t_0; Sx, h \mapsto t_S)/h] : \sigma[St/x]}\mathbb{N}\text{-C}$$

$$\frac{\begin{array}{c}\Gamma \vdash t : \mathbb{N} \qquad \Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma, x : \mathbb{N} \vdash u : \sigma \qquad \Gamma \vdash t_0 : \sigma[0/x] \\ \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x] \qquad \Gamma \vdash u[0/x] \equiv t_0 : \sigma[0/x] \qquad \Gamma \vdash u[Sx/x] \equiv t_S[u/h]\end{array}}{\Gamma \vdash u[t/x] \equiv \text{elim}^{\mathbb{N}}_{x.\sigma}(0 \mapsto t_0; Sx, h \mapsto t_S) : \sigma[t/x]}\mathbb{N}\text{-U}$$

Наконец, самый интересный тип (и первый по-настоящему зависимый тип) — тип равенства, благодаря которому внутри системы Мартина-Лёфа становится возможным доказывать всевозможные теоремы. Он индексируется двумя представителями одного и того же произвольного типа; единственный конструктор — доказательство по рефлексивности. Имея в виду этот факт, правило элиминации равенства позволяет переписывать одни термы внутри типов на другие, если доказано, что они равны.

$$\frac{\Gamma \vdash \sigma\,\text{type}}{\Gamma, x : \sigma, y : \sigma \vdash x = y\,\text{type}}{=}\text{-F} \qquad \frac{\Gamma \vdash \sigma\,\text{type}}{\Gamma, x : \sigma \vdash \text{refl}\, x : x = x}{=}\text{-I}$$

$$\frac{\Gamma, x : \sigma, y : \sigma, z : x = y \vdash \tau\,\text{type} \qquad \Gamma, x : \sigma \vdash u : \tau[x/y, \text{refl}\, x/z]}{\Gamma, x : \sigma, y : \sigma, z : x = y \vdash \text{elim}^{=}_{xyz.\tau}\, z(\text{refl}\, x \mapsto u) : \tau}{=}\text{-E}$$

$$\frac{\Gamma, x : \sigma, y : \sigma, z : x = y \vdash \tau\,\text{type} \qquad \Gamma, x : \sigma \vdash u : \tau[x/y, \text{refl}\, x/z]}{\Gamma, x : \sigma \vdash \text{elim}^{=}_{xyz.\tau}(\text{refl}\, x)(\text{refl}\, x \mapsto u) \equiv u : \tau[x/y, \text{refl}\, x/z]}{=}\text{-C}$$

2.2 Метатеоремы

2.3 Задача полной и частичной типизации

3 Операционная семантика

3.1 Семантика лямбда-исчисления

3.2 Стековая машина с кодовой секцией

Программа для стековой машины с кодовой секцией — пара (D, S), где S — это последовательность инструкций ("подпрограмма"), которые машине предстоит выполнить, а D — "кодовая секция", словарь подпрограмм, на которые можно ссылаться при выполнении любой подпрограммы.

При выполнении программы машина манипулирует стеком, в котором могут лежать как численные значения, так и имена подпрограмм из кодовой секции. Выполнение программы состоит из последовательного исполнения инструкций из S.

Каждая инструкция имеет вид $\texttt{op}(x_1, \ldots, x_a)$, где a — число значений, которое она забирает с верхушки стека для выполнения. Доступны следующие инструкции:

$n()$	Положить число n на верхушку стека.
$x()$	Положить имя x на верхушку стека.
$+(x_1, x_2)$	Заменить два верхних числа на стеке на их сумму.
$-(x_1, x_2)$	Аналогично, но числа вычитаются.
$(\cdot)(x_1, x_2)$	Аналогично, но числа перемножаются.
$\texttt{if0}(x_1, x_2, x_3)$	Если $x_1 = 0$, оставляет на верхушке x_2, иначе x_3.
$\texttt{copy}(x_1)$	Заменить число x_1 на верхушке на x_1-е число в стеке, считая снизу.
$\texttt{write}(x_1, x_2)$	Записать x_2 на x_1-ю позицию в стеке, считая снизу.
$\texttt{call}(x_1)$	Выполнить $D[x_1]$, затем вернуться к текущей подпрограмме.

3.3 Машина с кучей

$\texttt{new}^n(x_1, \ldots, x_n)$	Перемещает $x_1, \ldots, x_n$ на кучу.
$\texttt{delete}(x_1)$	Удаляет значения по указателю x_1 из кучи.

4 Линейные типы и ресурсная логика

4.1 Безопасное управление памятью

4.2 Линейные зависимые типы

Bunched contexts

Vakar:

$$\vdash \Delta; \Xi \text{ ctxt}$$
$$\Delta; \cdot \vdash \tau \text{ type}$$
$$\Delta; \Xi \vdash t : \tau$$
$$\vdash \Delta; \Xi \equiv \Delta'; \Xi' \text{ ctxt}$$
$$\Delta; \cdot \vdash \tau \equiv \tau' \text{ type}$$
$$\Delta; \Xi \vdash t \equiv t' : \tau$$

$$\frac{\vdash \Delta, x : \tau, \Delta'; \cdot \text{ ctxt}}{\Delta, x : \tau, \Delta'; \cdot \vdash x : \tau}\textsc{Int-Var} \qquad \frac{\vdash \Delta; x : \tau \text{ ctxt}}{\Delta; x : \tau \vdash x : \tau}\textsc{Lin-Var}$$

$$\frac{\Delta, x : \sigma, \Delta'; \cdot \vdash \tau \text{ type} \qquad \Delta; \cdot \vdash t : \sigma}{\Delta, \Delta'[t/x]; \cdot \vdash \tau[t/x] \text{ type}}\textsc{Int-Ty-Subst}$$

$$\frac{\Delta, x : \sigma, \Delta'; \Xi \vdash u : \tau \qquad \Delta; \cdot \vdash t : \sigma}{\Delta, \Delta'[t/x]; \Xi[t/x] \vdash u[t/x] : \tau[t/x]}\textsc{Int-Tm-Subst}$$

$$\frac{\Delta; \Xi, x : \sigma \vdash u : \tau \qquad \Delta; \Xi' \vdash t : \sigma}{\Delta; \Xi, \Xi' \vdash u[t/x] : \tau}\textsc{Lin-Tm-Subst}$$

$$\frac{\Delta; \cdot \vdash \sigma\,\mathrm{type} \qquad \Delta, x : \sigma; \cdot \vdash \tau\,\mathrm{type}}{\Delta; \cdot \vdash \Sigma_{!x:!\sigma}\tau\,\mathrm{type}}\Sigma\text{-F} \qquad \frac{\Delta; \cdot \vdash t : \sigma \qquad \Delta; \Xi \vdash u : \tau[t/x]}{\Delta; \Xi \vdash !t \otimes u : \Sigma_{!x:!\sigma}\tau}\Sigma\text{-I}$$

$$\frac{\Delta; \cdot \vdash \rho\,\mathrm{type} \qquad \Delta; \Xi \vdash t : \Sigma_{!x:!\sigma}\tau \qquad \Delta, x : \sigma; \Xi', y : \tau \vdash r : \rho}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\,!x \otimes y = t \text{ in } r : \rho}\Sigma\text{-E}$$

$$\frac{\Delta; \cdot \vdash \sigma\,\mathrm{type} \qquad \Delta; \cdot \vdash \tau\,\mathrm{type}}{\Delta; \cdot \vdash \sigma \multimap \tau\,\mathrm{type}}\multimap\text{-F} \qquad \frac{\Delta; \Xi, x : \sigma \vdash t : \tau}{\Delta; \Xi \vdash \lambda_{x:\sigma} t : \sigma \multimap \tau}\multimap\text{-I}$$

$$\frac{\Delta; \Xi \vdash f : \sigma \multimap \tau \qquad \Delta; \Xi' \vdash t : \sigma}{\Delta; \Xi, \Xi' \vdash f\ t : \tau}\multimap\text{-E} \qquad \frac{\Delta; \cdot \vdash \sigma\,\mathrm{type}}{\Delta; \cdot \vdash !\sigma\,\mathrm{type}}!\text{-F} \qquad \frac{\Delta; \cdot \vdash t : \sigma}{\Delta; \cdot \vdash !t : !\sigma}!\text{-I}$$

$$\frac{\Delta; \Xi \vdash t : !\sigma \qquad \Delta, x : \sigma; \Xi' \vdash u : \tau}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\,!x = t \text{ in } u : \tau}!\text{-E}$$

Аналогично вводятся Π-, 1-, $\otimes$-, $\top$-, &-, 0- и $\oplus$- типы.

5 Экзистенциальные типы

5.1 Эффективное представление замыканий

$$\frac{\Gamma, \blacksquare, x : \sigma; \Psi, \blacksquare, y : \tau \vdash t : \rho \qquad \blacksquare \notin \Gamma, \Psi \qquad x \notin \mathrm{Free}(\Psi)}{\Gamma; \Psi \vdash \lambda_{!x:!\sigma} \lambda_{y:\tau} t : \Pi_{!x:!\sigma} \tau \multimap \rho}$$

$$(\sigma \multimap' \tau) \coloneqq \Sigma_{!\gamma:!\,\mathrm{type}} \gamma \otimes (\sigma \otimes \gamma \multimap \tau)$$

$$\Pi'_{!x:!\sigma} \tau \coloneqq \Sigma_{!\gamma:!\,\mathrm{type}} \gamma \otimes (\Pi_{!x:!\sigma} \gamma \multimap \tau)$$

5.2 Вселенные

Парадокс Жирара!

5.3 Рефлективные подвселенные

$$\frac{\Delta; \cdot \vdash A\,\mathrm{type}_1}{\Delta; \cdot \vdash \bigcirc A\,\mathrm{type}_0}\bigcirc\text{-F} \qquad \frac{\Delta; \cdot \vdash A\,\mathrm{type}_1 \qquad \Delta; \Xi \vdash a : A}{\Delta; \Xi \vdash \eta_A a : \bigcirc A}\bigcirc\text{-I}$$

$$\frac{\Delta; \cdot \vdash C\,\mathrm{type}_O \qquad \Delta; \Xi \vdash u : \bigcirc A \qquad \Delta; x : A, \Xi' \vdash v : C}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\ \eta_A x = u\ \mathrm{in}\ v : C}\bigcirc\text{-E}$$

6 Прямой доступ к памяти

6.1 Прямой доступ к стеку

$$\frac{\Delta; \cdot \vdash A\,\mathrm{type}}{\Delta; \cdot \vdash *A\,\mathrm{type}}\,{*}\text{-F} \qquad \frac{\Delta; \cdot \vdash r : *A}{\Delta; \cdot \vdash \text{🔒}\,r\ \ \mathrm{type}}\,\text{🔒-F}$$

$$\frac{\Delta; \cdot \vdash C\,\mathrm{type} \qquad \Delta; \Xi \vdash a : A \qquad \Delta, r : *A; \Xi', l : \text{🔒}\,r \vdash c : C \otimes \text{🔒}\,r}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\,!r \otimes l = *a\ \mathrm{in}\ c : C}\,\textsc{StackVar}$$

$$\frac{\Delta; \Xi, x : A \vdash t : A \otimes C}{\Delta, r : *A; \Xi, l : \text{🔒}\,r \vdash \mathrm{modify}_r\ l\ \mathrm{with}\ (x \mapsto t) : C \otimes \text{🔒}\,r}\,\textsc{Modify}$$

6.2 Прямой доступ к куче

$$\frac{}{\Delta; \cdot \vdash \mathrm{alloc}\,\mathrm{type}}\,\text{alloc-F} \qquad \frac{\Delta; \Xi \vdash t : \tau}{\Delta; \Xi, a : \mathrm{alloc} \vdash \mathrm{new}_a\ t : \Sigma_{!r:!*\tau}\,\text{🔒}\,r \otimes \mathrm{alloc}}\,\textsc{New}$$

$$\frac{\Delta; \Xi \vdash r : *\tau \qquad \Delta, r : *\tau; \Xi' \vdash l : \text{🔒}\,r}{\Delta; \Xi, \Xi', a : \mathrm{alloc} \vdash \mathrm{free}_{r,a}\ l : A \otimes \mathrm{alloc}}\,\textsc{Free}$$

7 Защищённая рекурсия

8 Исчисление указателей

8.1 Логический фреймворк

$$\frac{\Delta; \cdot \vdash A \Rightarrow \mathrm{type}_i}{\Delta; \cdot \vdash \triangleright A \Rightarrow \mathrm{type}_i} \triangleright\text{-F} \qquad \frac{\Delta, * \triangleright X : \mathrm{type}_0; \cdot \vdash F \Rightarrow \mathrm{type}_0}{\Delta; \cdot \vdash \mu X.F \Rightarrow \mathrm{type}_0} \mu\text{-F}$$

8.2 Метатеоремы

8.3 Алгоритм частичной типизации

9 Дальнейшие расширения

9.1 Строковый полиморфизм

9.2 Полиморфизм уровней

9.3 Аллокации арен

9.4 Многопоточное исполнение и модель памяти

9.5 Двухуровневая система типов

10 Заключение

Список литературы

[1] AspenCore Media. *Embedded: News & Resources For The Electronics Community.* URL: https://www.embedded.com/ (дата обр. 31.05.2024).

[2] Copyright Free Software Foundation, Inc. *GDB: The GNU Project Debugger.* 2024. URL: https://www.sourceware.org/gdb/ (дата обр. 31.05.2024).

[3] Mart W. M. Graef. «More Than Moore White Paper». В: *2021 IEEE International Roadmap for Devices and Systems Outbriefs* (2021), с. 1—47.

[4] Adam Hayes. *Smart Home: Definition, How They Work, Pros and Cons.* 2024. URL: https://www.investopedia.com/terms/s/smart-home.asp (дата обр. 31.05.2024).

[5] Daniel Holden, Claudio Daffra и др. *tgc — A Tiny Garbage Collector for C.* 2023. URL: https://github.com/orangeduck/tgc (дата обр. 31.05.2024).

[6] ISO/IEC. *ISO International Standard ISO/IEC 14882:2020(E) – Programming Language C++.* Geneva, Switzerland: International Organization for Standardization (ISO), 2020. URL: https://isocpp.org/std/the-standard (дата обр. 31.05.2024).

[7] Kuruva Lakshmanna и др. «The Rise of "Internet of Things": Review and Open Research Issues Related to Detection and Prevention of IoT-Based Security Attacks». В: *Wireless Communications and Mobile Computing* (2022). DOI: 10.1155/2022/8669348.

[8] John Launchbury и Simon L. Peyton Jones. «Lazy functional state threads». В: *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation.* PLDI '94. Orlando, Florida, USA: Association for Computing Machinery, 1994, с. 24—35. ISBN: 089791662X. DOI: 10.1145/178243.178246.

[9] Nicholas Nethercote и Julian Seward. «Valgrind: a framework for heavyweight dynamic binary instrumentation». В: *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation.* PLDI '07. San Diego, California, USA: Association for Computing Machinery, 2007, с. 89—100. ISBN: 9781595936332. DOI: 10.1145/1250734.1250746.

[10] Benjamin C. Pierce. *Types and Programming Languages.* 1-е изд. MIT Press, февр. 2002. ISBN: 0262162091.

[11] PVS-Studio LLC. *PVS-Studio is a solution to enhance code quality, security (SAST), and safety.* 2008-2024. URL: https://pvs-studio.com/en/ (дата обр. 31.05.2024).

[12] Системное программное обеспечение. *Системное программное обеспечение — Википедия, свободная энциклопедия.* 2023. URL: https://ru.wikipedia.org/wiki/%D0%A1%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D0%BD%D0%BE%D0%B5_%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%BD%D0%BE%D0%B5_%D0%BE%D0%B1%D0%B5%D1%81%D0%BF%D0%B5%D1%87%D0%B5%D0%BD%D0%B8%D0%B5 (дата обр. 31.05.2024).

[13] Ислам Талипов. *alien-memory — A Scala library for type-safe direct memory access via Foreign Functions & Memory API.* 2024. URL: https://yandexclassifieds.github.io/alien-memory/ (дата обр. 31.05.2024).